Python Activity-Based Learning

1.

Retail Billing with Multiple Discount Rules: Compute item-wise subtotal, apply tiered discount rules with a maximum cap, add GST, and generate a formatted final bill.

Sol:

Problem statement

Calculate a customer's retail subtotal across multiple items, apply a 10% discount above Rs. 2,000 capped at Rs. 500, add 5% GST, and display the final bill.

Objectives

- Read price and quantity for each item and compute the running subtotal.
- Apply a 10% discount when subtotal exceeds Rs. 2,000 with a maximum cap of Rs. 500.
- Calculate 5% GST on the discounted amount and display the final bill summary.

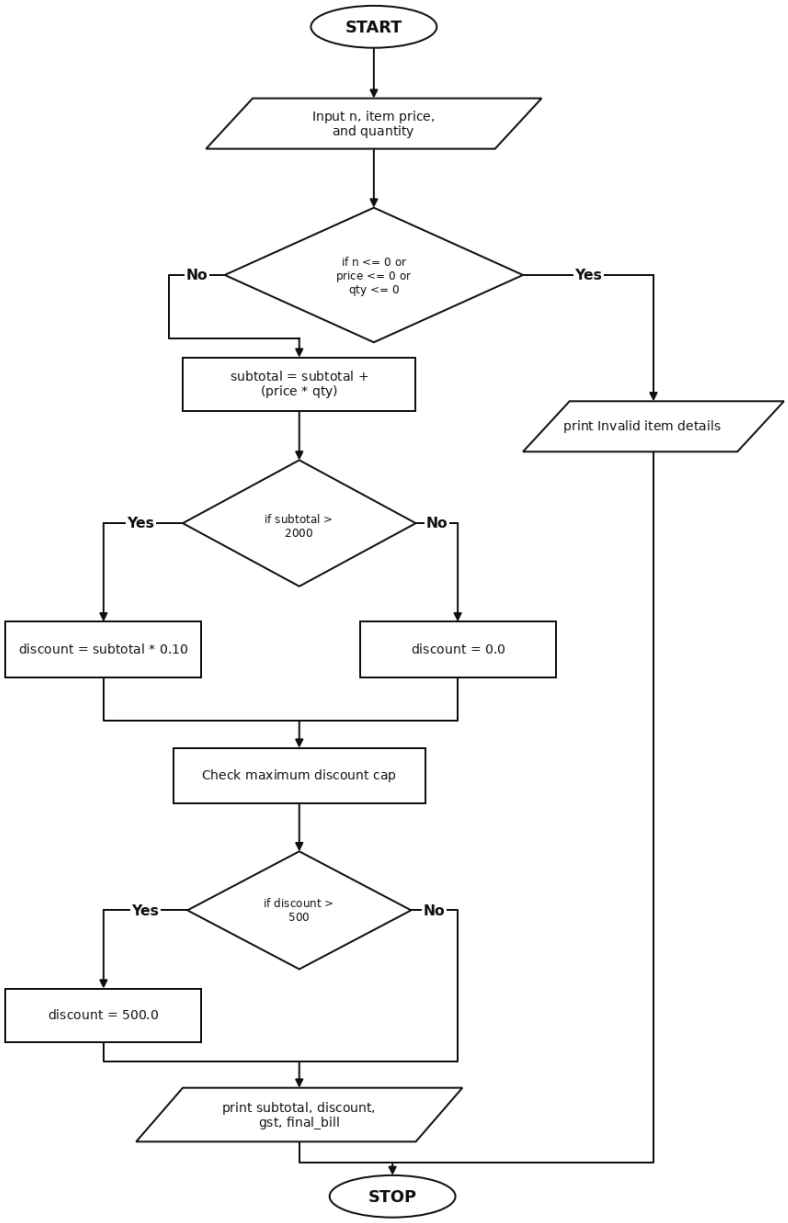
Assumptions

- Number of items, item price, and item quantity must all be greater than zero.
- Orders above Rs. 2,000 receive a 10% discount up to a maximum discount of Rs. 500.
- GST is charged at 5% on the amount after discount.

Algorithm

1. Start.
2. Read the number of items n and initialize subtotal = 0.0.
3. For each item from 1 to n, read item price and quantity.
4. Validate that price > 0 and qty > 0; otherwise print error and stop loop.
5. Add price * qty to subtotal.
6. If valid, compute 10% discount for subtotal > 2000 (capped at 500) and 5% GST.
7. Display subtotal, discount, GST, and final bill, then stop.

Flowchart



Pseudocode

START

INPUT n

SET subtotal = 0.0, valid = (n > 0)

FOR i = 1 TO n

  INPUT price, qty

  IF price <= 0 OR qty <= 0

    PRINT "Invalid item details"

```
        SET valid = False
        BREAK
    subtotal = subtotal + (price * qty)
IF valid == True
    IF subtotal > 2000
        discount = subtotal * 0.10
        IF discount > 500
            discount = 500.0
    ELSE
        discount = 0.0
    after_discount = subtotal - discount
    gst = after_discount * 0.05
    final_bill = after_discount + gst
    PRINT subtotal, discount, gst, final_bill
STOP
```

Source Code

```
n = int(input("Enter number of items: "))
subtotal = 0.0
valid = n > 0

for i in range(n):
    price = float(input("Enter price of item " + str(i + 1) + ": "))
    qty = int(input("Enter quantity of item " + str(i + 1) + ": "))
    if price <= 0 or qty <= 0:
        print("Invalid item details")
        valid = False
        break
    subtotal = subtotal + (price * qty)

if valid:
    if subtotal > 2000:
        discount = subtotal * 0.10
        if discount > 500:
            discount = 500.0
    else:
        discount = 0.0
```

```
after_discount = subtotal - discount
gst = after_discount * 0.05
final_bill = after_discount + gst

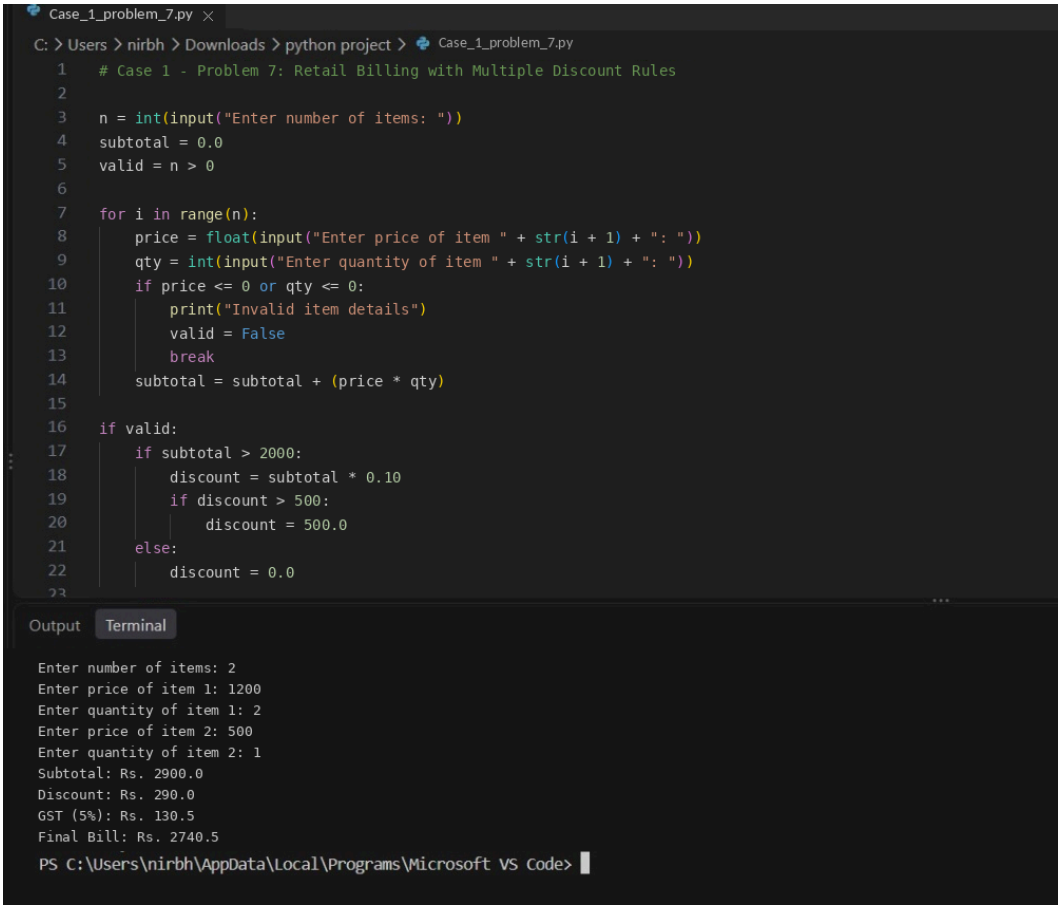
print("Subtotal: Rs.", round(subtotal, 2))
print("Discount: Rs.", round(discount, 2))
print("GST (5%): Rs.", round(gst, 2))
print("Final Bill: Rs.", round(final_bill, 2))
```

Test Cases

1. Enter number of items: 2
   Enter price of item 1: 1200
   Enter quantity of item 1: 2
   Enter price of item 2: 500
   Enter quantity of item 2: 1
   Output:
   Subtotal: Rs. 2900.0
   Discount: Rs. 290.0
   GST (5%): Rs. 130.5
   Final Bill: Rs. 2740.5

2. Enter number of items: 2
   Enter price of item 1: 3000
   Enter quantity of item 1: 2
   Enter price of item 2: 1000
   Enter quantity of item 2: 1
   Output:
   Subtotal: Rs. 7000.0
   Discount: Rs. 500.0
   GST (5%): Rs. 325.0
   Final Bill: Rs. 6825.0

Output



Conclusion

The program calculates the retail item subtotal, applies capped discount and GST rules, and prints the final bill.

2.

Manufacturing Cost and Profitability Analysis: Calculate total production cost including material wastage, compute revenue and profit percentage, and evaluate profitability target status.

Sol:

Problem statement

Compute total manufacturing cost with 5% material wastage, calculate net profit and profit percentage from unit sales, and check if the 15% profit target is achieved.

Objectives

- Calculate material wastage and total production cost from material, labour, and machine costs.
- Compute total revenue, net profit, and profit percentage.
- Classify production profitability against the 15% profit target threshold.

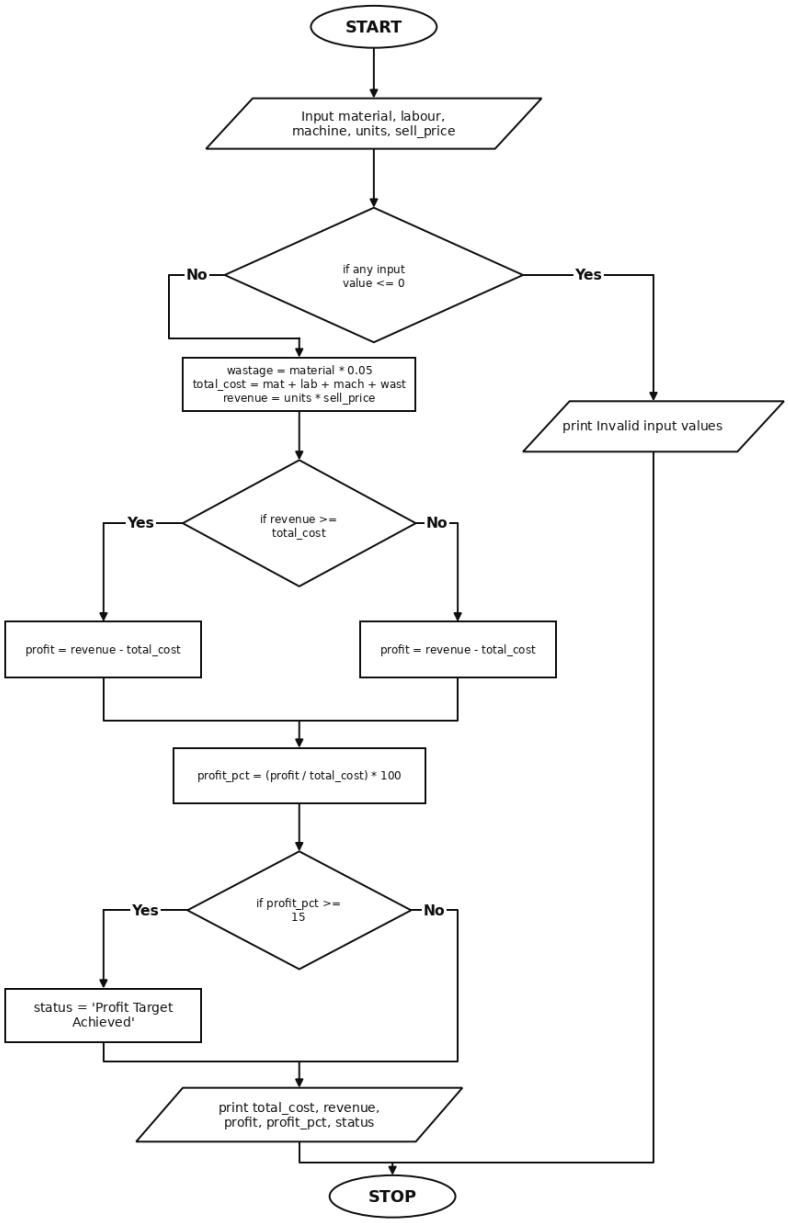
Assumptions

- Material, labour, machine costs, units produced, and selling price must be greater than zero.
- Material wastage is fixed at 5% of material cost.
- Minimum profit percentage required to achieve the target is 15%.

Algorithm

1. Start.
2. Read material cost, labour cost, machine cost, units produced, and selling price per unit.
3. Validate that all input values are greater than zero.
4. Calculate wastage = material * 0.05 and total_cost = material + labour + machine + wastage.
5. Compute revenue = units * sell_price, profit = revenue - total_cost, and profit_pct.
6. Check if profit_pct >= 15 to assign the profitability status.
7. Display cost, revenue, net profit, profit percentage, and status, then stop.

Flowchart



Pseudocode

START

INPUT material, labour, machine, units, sell_price

IF material <= 0 OR labour <= 0 OR machine <= 0 OR units <= 0 OR sell_price <= 0

  PRINT "Invalid input values"

ELSE

  wastage = material * 0.05

  total_cost = material + labour + machine + wastage

```
    revenue = units * sell_price
    profit = revenue - total_cost
    profit_pct = (profit / total_cost) * 100
    IF profit_pct >= 15
        status = "Profit Target Achieved"
    ELSE
        status = "Below Profit Target"
    PRINT total_cost, revenue, profit, profit_pct, status
STOP
```

Source Code

```
material = float(input("Enter Material Cost: "))
labour = float(input("Enter Labour Cost: "))
machine = float(input("Enter Machine Cost: "))
units = int(input("Enter Units Produced: "))
sell_price = float(input("Enter Selling Price per Unit: "))

if material <= 0 or labour <= 0 or machine <= 0 or units <= 0 or sell_price <= 0:
    print("Invalid input values")
else:
    wastage = material * 0.05
    total_cost = material + labour + machine + wastage
    revenue = units * sell_price
    profit = revenue - total_cost
    profit_pct = (profit / total_cost) * 100

    if profit_pct >= 15:
        status = "Profit Target Achieved"
    else:
        status = "Below Profit Target"

    print("Total Production Cost: Rs.", round(total_cost, 2))
    print("Total Revenue: Rs.", round(revenue, 2))
    print("Net Profit: Rs.", round(profit, 2))
    print("Profit Percentage:", round(profit_pct, 2), "%")
    print("Status:", status)
```

Test Cases

1. Enter Material Cost: 20000

   Enter Labour Cost: 12000

   Enter Machine Cost: 7000

   Enter Units Produced: 500

   Enter Selling Price per Unit: 100

   Output:

   Total Production Cost: Rs. 40000.0

   Total Revenue: Rs. 50000.0

   Net Profit: Rs. 10000.0

   Profit Percentage: 25.0 %

   Status: Profit Target Achieved

2. Enter Material Cost: 10000

   Enter Labour Cost: 8000

   Enter Machine Cost: 4500

   Enter Units Produced: 250

   Enter Selling Price per Unit: 100

   Output:

   Total Production Cost: Rs. 23000.0

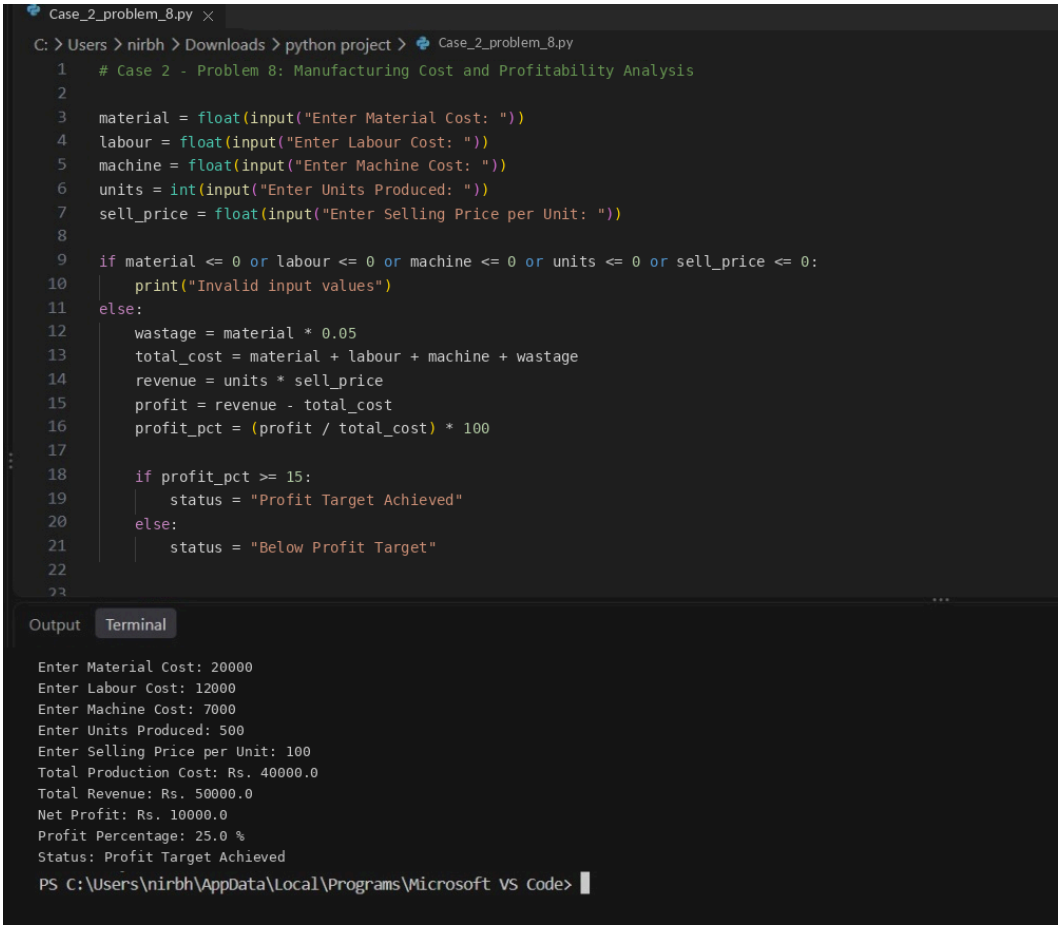
   Total Revenue: Rs. 25000.0

   Net Profit: Rs. 2000.0

   Profit Percentage: 8.7 %

   Status: Below Profit Target

Output



Conclusion

The program computes manufacturing cost with material wastage, evaluates profit percentage, and reports whether the profit target is met.

3.

Fuel Consumption and Route Comparison: Estimate fuel requirements with buffer allowance across multiple vehicle types, compute trip fuel costs, and identify the lowest-cost vehicle option.

Sol:

Problem statement

Compare fuel consumption and trip cost across Hatchback, Sedan, and SUV vehicle categories with a 10% fuel buffer and determine the most economical option.

Objectives

- Calculate required fuel for each vehicle type including a 10% route buffer.
- Compute trip fuel cost for Hatchback, Sedan, and SUV options.
- Identify and display the vehicle option with the minimum trip cost.

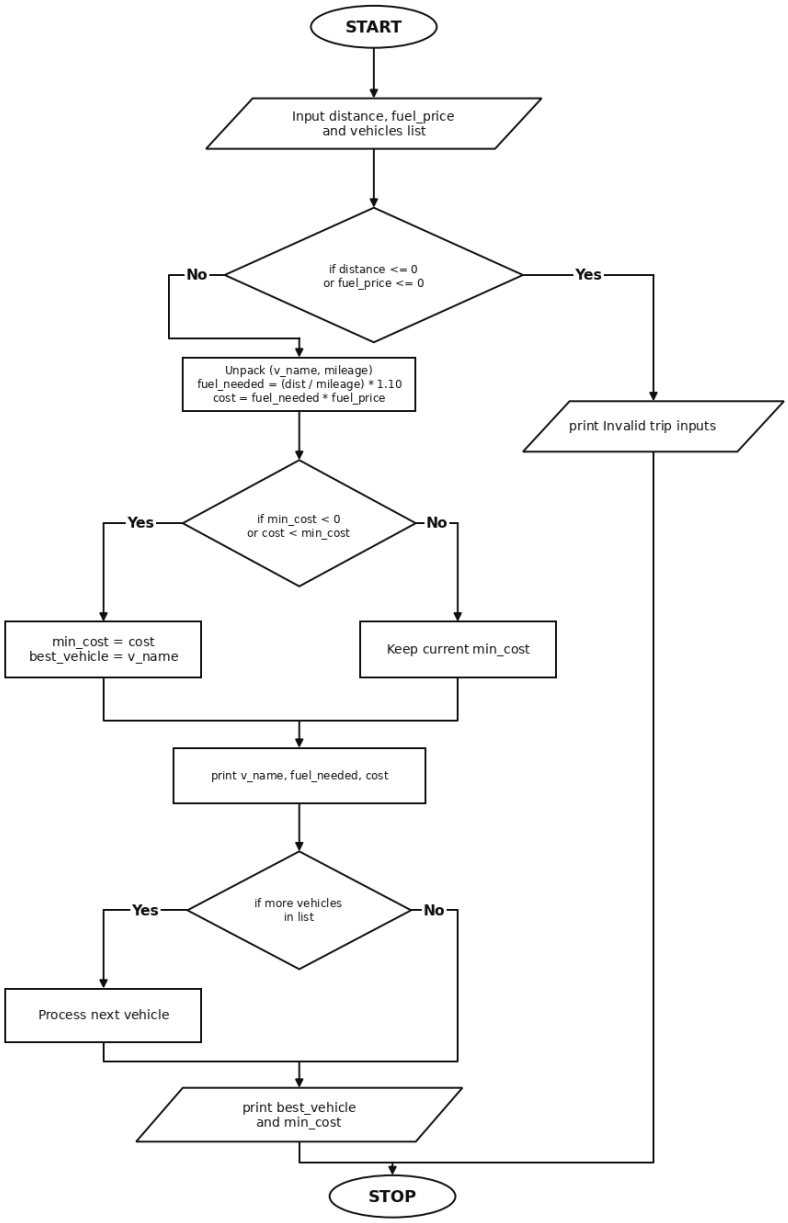
Assumptions

- Trip distance and fuel price per litre must be greater than zero.
- Mileage rates are 20.0 km/L for Hatchback, 15.0 km/L for Sedan, and 12.0 km/L for SUV.
- Actual fuel required includes a 10% buffer (multiplied by 1.10).

Algorithm

1. Start.
2. Read trip distance in km and fuel price per litre.
3. Define the list of vehicle tuples (Hatchback, Sedan, SUV) with their mileage values.
4. Validate that distance > 0 and fuel_price > 0.
5. For each vehicle, compute fuel_needed = (distance / mileage) * 1.10 and cost = fuel_needed * fuel_price.
6. Track the vehicle with the lowest fuel cost.
7. Display vehicle-wise fuel and cost details along with the lowest cost option, then stop.

Flowchart



Pseudocode

START

INPUT distance, fuel_price

IF distance <= 0 OR fuel_price <= 0

    PRINT "Invalid trip inputs"

ELSE

    SET best_vehicle = "", min_cost = -1.0

    FOR each (v_name, mileage) IN vehicles

```
        fuel_needed = (distance / mileage) * 1.10
        cost = fuel_needed * fuel_price
        PRINT v_name, fuel_needed, cost
        IF min_cost < 0 OR cost < min_cost
            min_cost = cost
            best_vehicle = v_name
    PRINT best_vehicle, min_cost
STOP
```

Source Code

```
distance = float(input("Enter Trip Distance (km): "))
fuel_price = float(input("Enter Fuel Price per Litre: "))

vehicles = [
    ("Hatchback", 20.0),
    ("Sedan", 15.0),
    ("SUV", 12.0)
]

if distance <= 0 or fuel_price <= 0:
    print("Invalid trip inputs")
else:
    best_vehicle = ""
    min_cost = -1.0

    for v in vehicles:
        v_name, mileage = v
        fuel_needed = (distance / mileage) * 1.10
        cost = fuel_needed * fuel_price
        print(v_name, "-> Fuel:", round(fuel_needed, 2), "L | Cost: Rs.", round(cost, 2))

        if min_cost < 0 or cost < min_cost:
            min_cost = cost
            best_vehicle = v_name

    print("Lowest Cost Option:", best_vehicle, "at Rs.", round(min_cost, 2))
```

Test Cases

1. Enter Trip Distance (km): 300

   Enter Fuel Price per Litre: 105

   Output:

   Hatchback -> Fuel: 16.5 L | Cost: Rs. 1732.5

   Sedan -> Fuel: 22.0 L | Cost: Rs. 2310.0

   SUV -> Fuel: 27.5 L | Cost: Rs. 2887.5

   Lowest Cost Option: Hatchback at Rs. 1732.5

2. Enter Trip Distance (km): 180

   Enter Fuel Price per Litre: 100

   Output:

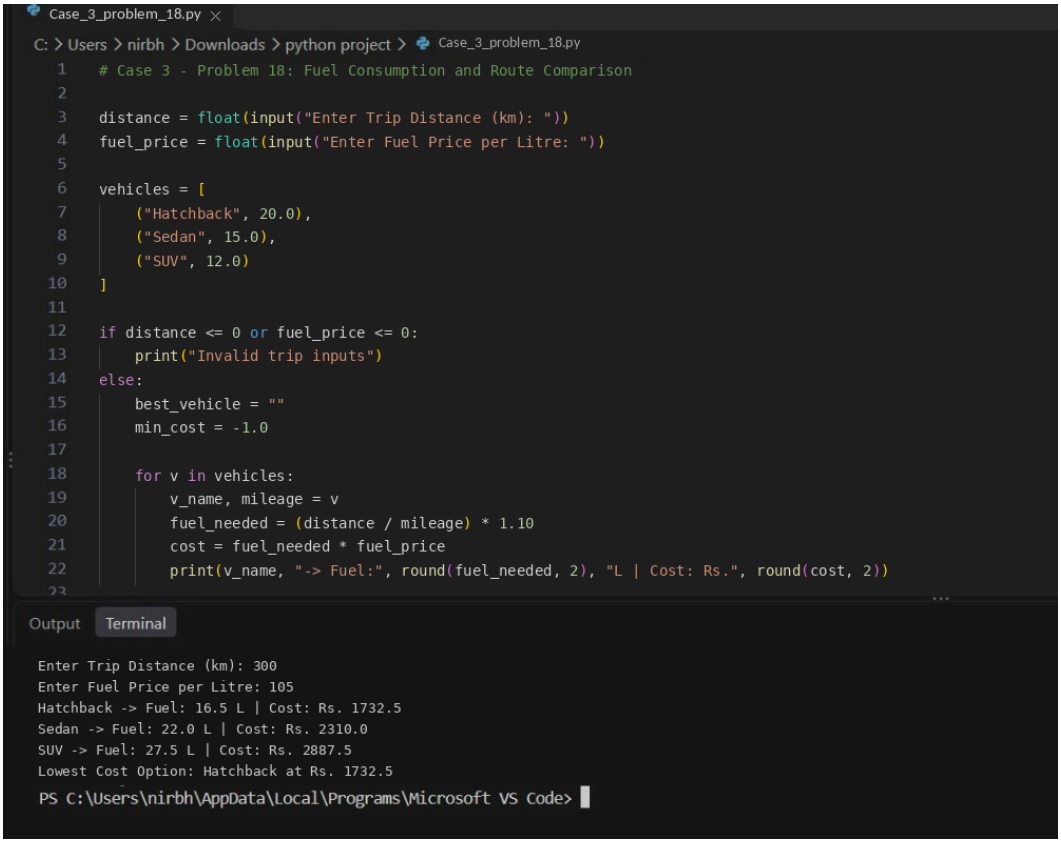
   Hatchback -> Fuel: 9.9 L | Cost: Rs. 990.0

   Sedan -> Fuel: 13.2 L | Cost: Rs. 1320.0

   SUV -> Fuel: 16.5 L | Cost: Rs. 1650.0

   Lowest Cost Option: Hatchback at Rs. 990.0

Output



Conclusion

The program compares fuel consumption and trip costs across vehicle types using tuple unpacking and identifies the lowest-cost option.

4.

Multi-Stage Examination Eligibility System: Evaluate student attendance, internal marks, laboratory completion, and tuition fee clearance to determine final examination eligibility and list unmet criteria.

Sol:

Problem statement

Verify whether a student meets all four examination eligibility criteria (attendance, internal marks, lab completion, and fee clearance) and list any unmet requirements.

Objectives

- Validate attendance percentage and internal marks within the 0 to 100 range.
- Check each eligibility requirement and record unmet conditions in a list.
- Display final examination eligibility status and all unmet requirements.

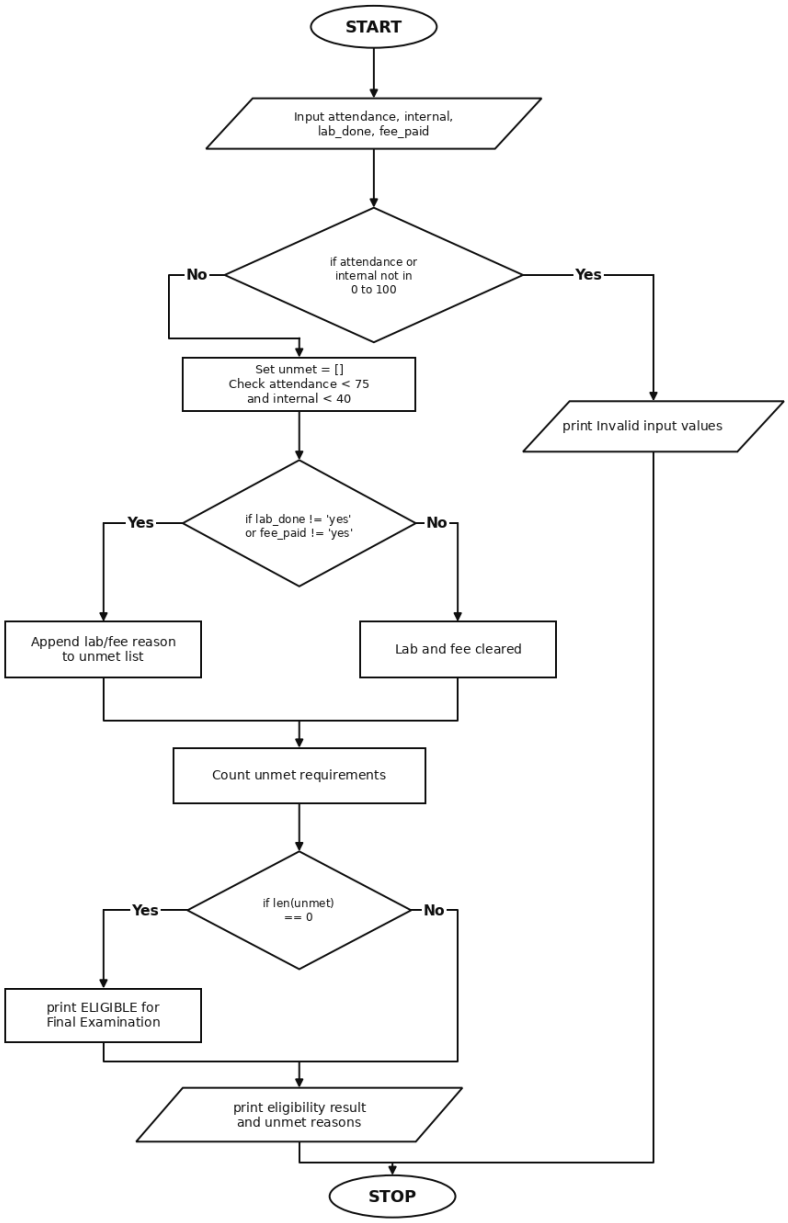
Assumptions

- Minimum required attendance is 75% and minimum internal score is 40 marks.
- Both laboratory work completion and tuition fee clearance must be answered as 'yes'.
- A student is eligible only when zero requirements remain unmet.

Algorithm

1. Start.
2. Read attendance percentage, internal marks, lab completion status, and fee clearance status.
3. Validate that attendance and internal marks are between 0 and 100.
4. Initialize an empty list unmet = [] and append each violated condition.
5. Check if len(unmet) == 0.
6. If empty, print that the student is eligible; otherwise print not eligible and list each reason.
7. Stop.

Flowchart



Pseudocode

START

INPUT attendance, internal, lab_done, fee_paid

IF attendance < 0 OR attendance > 100 OR internal < 0 OR internal > 100

  PRINT "Invalid input values"

ELSE

  SET unmet = []

  IF attendance < 75

```
        Append "Attendance below 75%" to unmet
    IF internal < 40
        Append "Internal marks below 40" to unmet
    IF lab_done != "yes"
        Append "Laboratory work incomplete" to unmet
    IF fee_paid != "yes"
        Append "Tuition fee not cleared" to unmet
    IF len(unmet) == 0
        PRINT "Result: ELIGIBLE for Final Examination"
    ELSE
        PRINT "Result: NOT ELIGIBLE" and unmet list
STOP
```

Source Code

```
attendance = float(input("Enter Attendance Percentage: "))
internal = float(input("Enter Internal Marks (0-100): "))
lab_done = input("Lab Completed? (yes/no): ").strip().lower()
fee_paid = input("Fee Cleared? (yes/no): ").strip().lower()

if attendance < 0 or attendance > 100 or internal < 0 or internal > 100:
    print("Invalid input values")
else:
    unmet = []
    if attendance < 75:
        unmet.append("Attendance below 75%")
    if internal < 40:
        unmet.append("Internal marks below 40")
    if lab_done != "yes":
        unmet.append("Laboratory work incomplete")
    if fee_paid != "yes":
        unmet.append("Tuition fee not cleared")

    if len(unmet) == 0:
        print("Result: ELIGIBLE for Final Examination")
    else:
        print("Result: NOT ELIGIBLE")
        print("Unmet Requirements:")
        for reason in unmet:
```

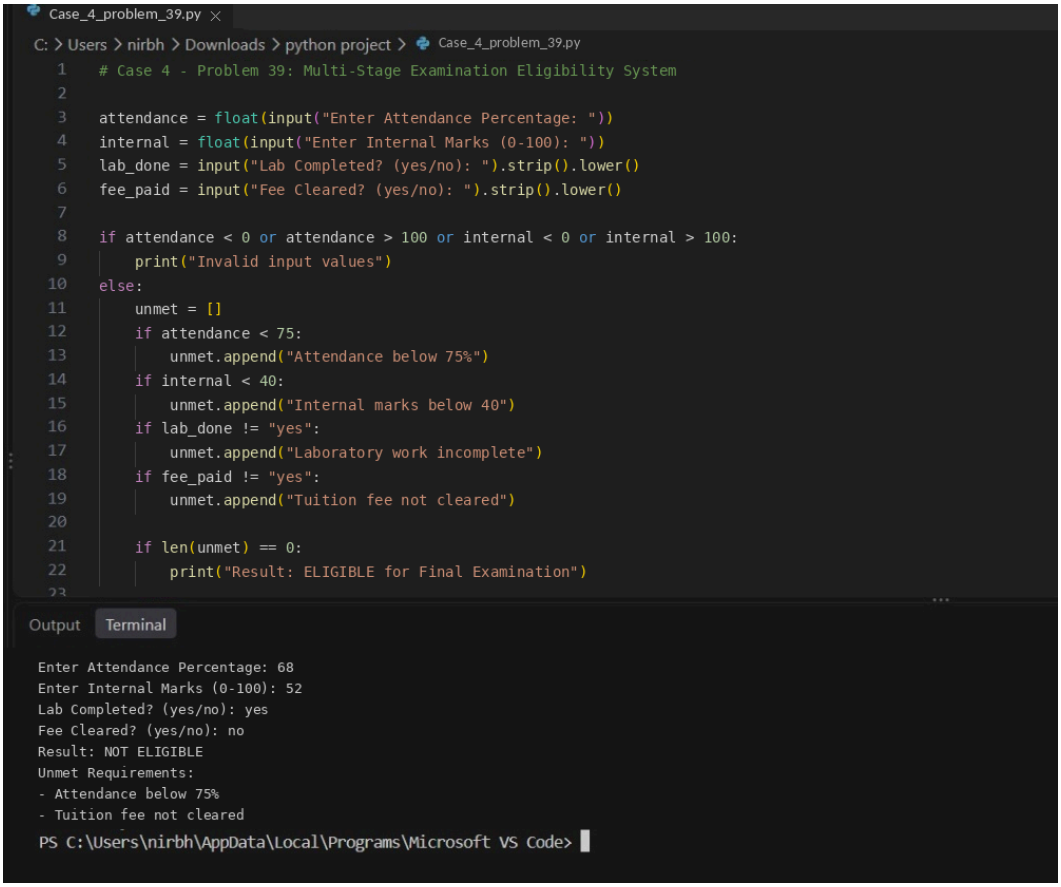
```
        print("-", reason)
```

Test Cases

1. Enter Attendance Percentage: 82
   Enter Internal Marks (0-100): 65
   Lab Completed? (yes/no): yes
   Fee Cleared? (yes/no): yes
   Output:
   Result: ELIGIBLE for Final Examination

2. Enter Attendance Percentage: 68
   Enter Internal Marks (0-100): 52
   Lab Completed? (yes/no): yes
   Fee Cleared? (yes/no): no
   Output:
   Result: NOT ELIGIBLE
   Unmet Requirements:
   - Attendance below 75%
   - Tuition fee not cleared

Output



Conclusion

The program evaluates multi-stage academic and administrative criteria and lists specific unmet requirements when a student is not eligible.

5.

Smart Parking Fee Management: Validate entry and exit hours, compute parking duration by vehicle category, apply hourly rates and daily maximum caps, and display total parking charges.

Sol:

Problem statement

Calculate parking duration and total parking fee for a car or bike based on 24-hour entry and exit times while enforcing vehicle-specific maximum fee limits.

Objectives

- Validate entry hour, exit hour, and vehicle type inputs.
- Compute parking duration and apply hourly tariff rates by vehicle type.
- Enforce the maximum daily fee cap and display the final parking charge.

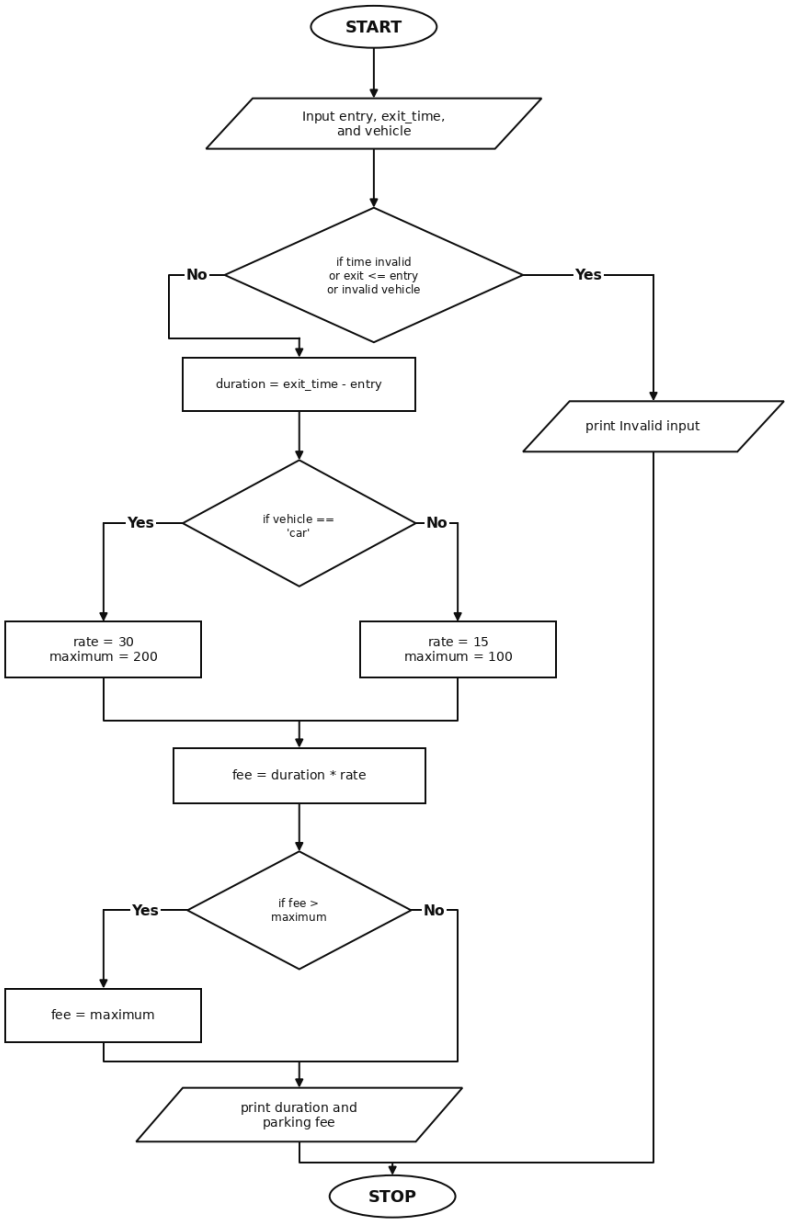
Assumptions

- Entry and exit hours are integers between 0 and 23 with exit hour greater than entry hour.
- Car parking costs Rs. 30/hour up to a maximum of Rs. 200.
- Bike parking costs Rs. 15/hour up to a maximum of Rs. 100.

Algorithm

1. Start.
2. Read entry hour, exit hour, and vehicle type (car/bike).
3. Validate time bounds (0-23), exit_time > entry, and vehicle in ['car', 'bike'].
4. Compute duration = exit_time - entry.
5. Set rate and maximum cap according to vehicle type.
6. Calculate fee = duration * rate and cap it at maximum if fee > maximum.
7. Display parking duration and parking fee, then stop.

Flowchart



Pseudocode

START

INPUT entry, exit_time, vehicle

IF entry < 0 OR entry > 23 OR exit_time < 0 OR exit_time > 23

    PRINT "Invalid time"

ELSE IF exit_time <= entry

    PRINT "Invalid exit time"

ELSE IF vehicle NOT IN ["car", "bike"]

```
    PRINT "Invalid vehicle"
ELSE
    duration = exit_time - entry
    IF vehicle == "car"
        rate = 30, maximum = 200
    ELSE
        rate = 15, maximum = 100
    fee = duration * rate
    IF fee > maximum
        fee = maximum
    PRINT duration, fee
STOP
```

Source Code

```
entry = int(input("Entry hour (0-23): "))
exit_time = int(input("Exit hour (0-23): "))
vehicle = input("Vehicle (car/bike): ").strip().lower()

if entry < 0 or entry > 23 or exit_time < 0 or exit_time > 23:
    print("Invalid time")
elif exit_time <= entry:
    print("Invalid exit time")
elif vehicle not in ["car", "bike"]:
    print("Invalid vehicle")
else:
    duration = exit_time - entry
    if vehicle == "car":
        rate = 30
        maximum = 200
    else:
        rate = 15
        maximum = 100

    fee = duration * rate
    if fee > maximum:
        fee = maximum

    print("Parking duration:", duration, "hours")
```

print("Parking fee: Rs.", fee)

Test Cases

1. Entry hour (0-23): 9

   Exit hour (0-23): 14

   Vehicle (car/bike): car

   Output:

   Parking duration: 5 hours

   Parking fee: Rs. 150

2. Entry hour (0-23): 8

   Exit hour (0-23): 18

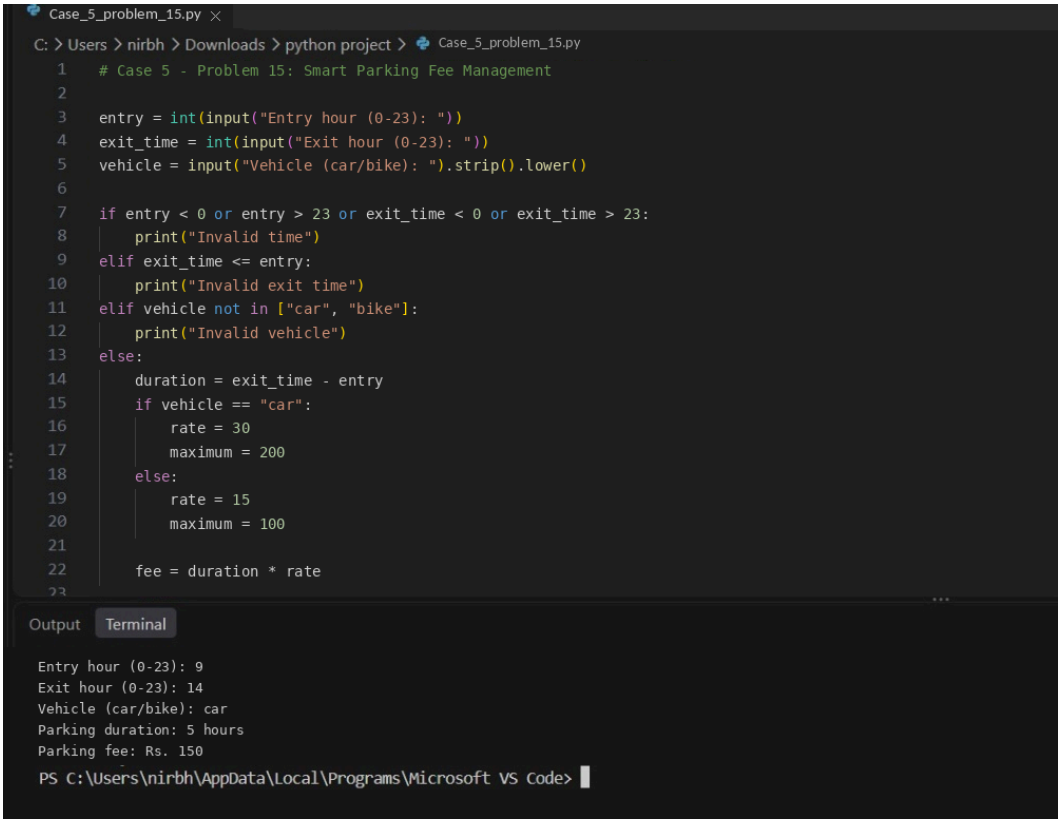
   Vehicle (car/bike): bike

   Output:

   Parking duration: 10 hours

   Parking fee: Rs. 100

Output



Conclusion

The program validates parking timestamps and vehicle categories, calculates hourly charges, and enforces maximum fee caps.

6.

Student Performance Trend Analysis: Compare earlier and latest marks across subjects, track score changes and subject pass status, and classify overall student performance from average marks.

Sol:

Problem statement

Analyze a student's mark changes between earlier and latest assessments across Python, Mathematics, and English, identify weak subjects, and classify overall performance.

Objectives

- Read earlier and latest marks for each subject and validate mark ranges.
- Compute score change and check if the latest score meets the 40-mark pass threshold.
- Calculate average latest marks and classify performance as Good or Needs Improvement.

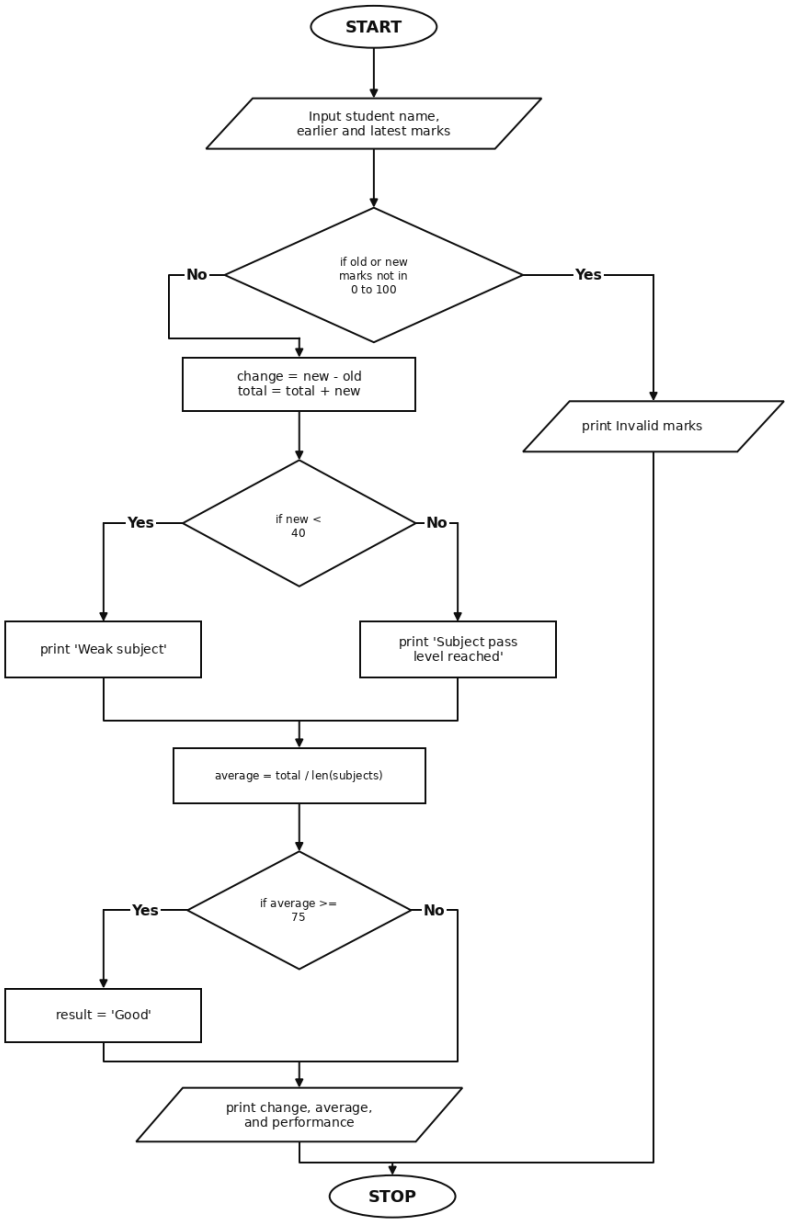
Assumptions

- Marks in all subjects must lie within the valid range of 0 to 100.
- Minimum passing score in each individual subject is 40 marks.
- An average of 75 or above is classified as Good; otherwise Needs Improvement.

Algorithm

1. Start.
2. Read student name and initialize subjects list, total = 0, and valid = True.
3. For each subject, read earlier marks (old) and latest marks (new).
4. If old or new is outside 0 to 100, print 'Invalid marks', set valid = False, and break.
5. Compute change = new - old, add new to total, and print change.
6. Print 'Weak subject' if new < 40, else 'Subject pass level reached'.
7. If valid, compute average = total / len(subjects) and classify performance.
8. Display average and performance status, then stop.

Flowchart



Pseudocode

START

INPUT name

SET subjects = ["Python", "Mathematics", "English"], total = 0, valid = True

FOR each subject IN subjects

  INPUT old, new

  IF old < 0 OR old > 100 OR new < 0 OR new > 100

    PRINT "Invalid marks"

```
        SET valid = False
        BREAK
    change = new - old
    total = total + new
    PRINT change and pass/weak status
IF valid == True
    average = total / len(subjects)
    IF average >= 75
        result = "Good"
    ELSE
        result = "Needs Improvement"
    PRINT average, result
STOP
```

Source Code

```
name = input("Student name: ")
subjects = ["Python", "Mathematics", "English"]
total = 0
valid = True

for subject in subjects:
    print(subject)
    old = int(input("Earlier marks: "))
    new = int(input("Latest marks: "))
    if old < 0 or old > 100 or new < 0 or new > 100:
        valid = False
        print("Invalid marks")
        break

    change = new - old
    total = total + new
    print("Change:", change)
    if new < 40:
        print("Weak subject")
    else:
        print("Subject pass level reached")

if valid:
```

```
average = total / len(subjects)
if average >= 75:
    result = "Good"
else:
    result = "Needs Improvement"
print("Average:", round(average, 2))
print("Performance:", result)
```

Test Cases

1. Student name: Rahul

   Python -> Earlier marks: 70, Latest marks: 82

   Mathematics -> Earlier marks: 65, Latest marks: 78

   English -> Earlier marks: 72, Latest marks: 80

   Output:

   Python -> Change: 12 | Subject pass level reached

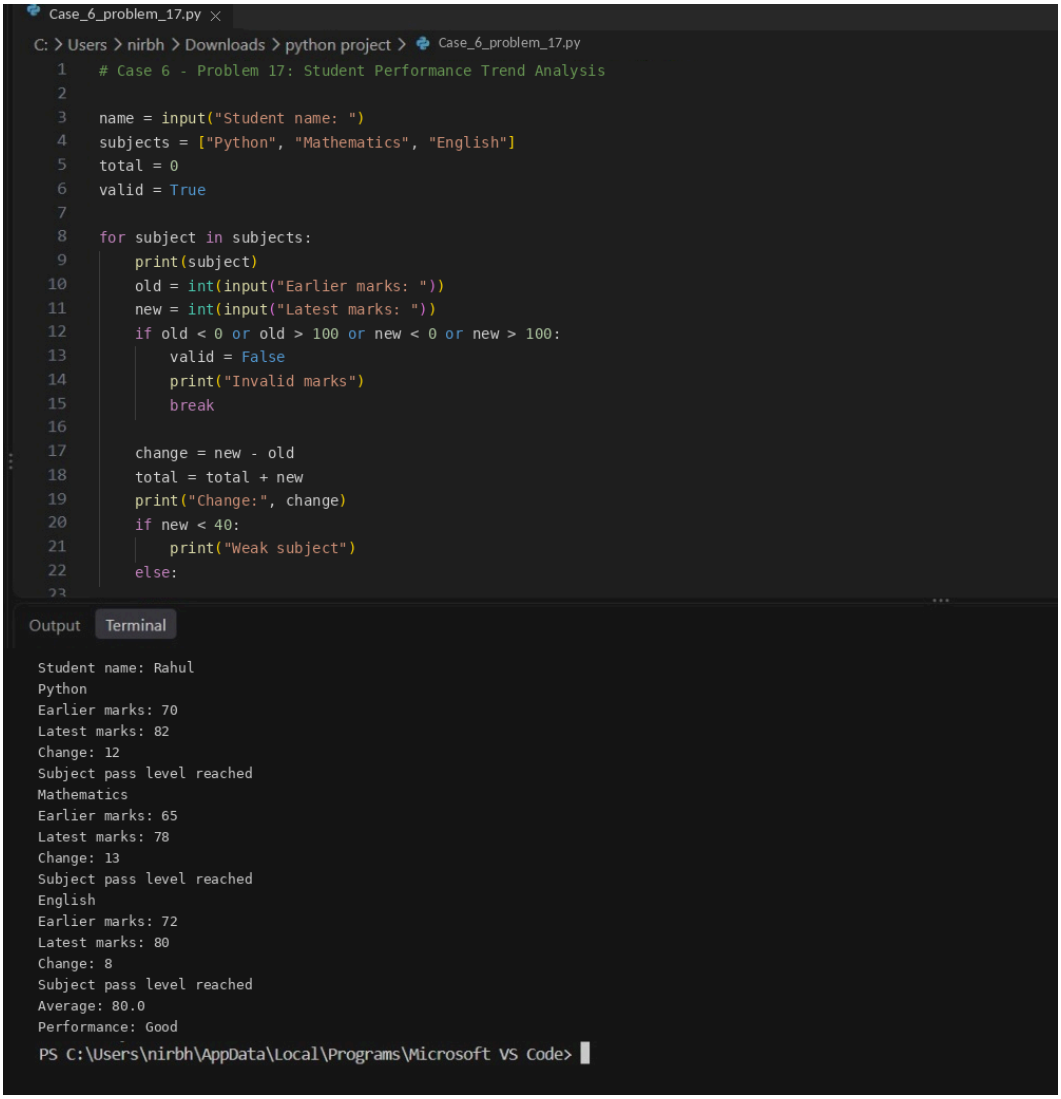
   Mathematics -> Change: 13 | Subject pass level reached

   English -> Change: 8 | Subject pass level reached

   Average: 80.0

   Performance: Good

Output



Conclusion

The program tracks subject-wise mark improvements, flags weak subjects, and evaluates overall student performance from the latest average.

7.

Courier Charge Optimization: Validate parcel dimensions, weight, distance zone, and delivery urgency, compare standard and express service charges, and select the optimal courier service.

Sol:

Problem statement

Validate parcel weight, dimensions, delivery zone, and urgency, calculate Standard and Express shipping tariffs for parcels up to 5 kg, and select the lower-cost courier service.

Objectives

- Validate parcel weight, dimensions, distance zone (1-3), and urgency option.
- Compute Standard and Express delivery charges including urgent surcharge.
- Enforce the 5 kg service limit and display the selected service and charge.

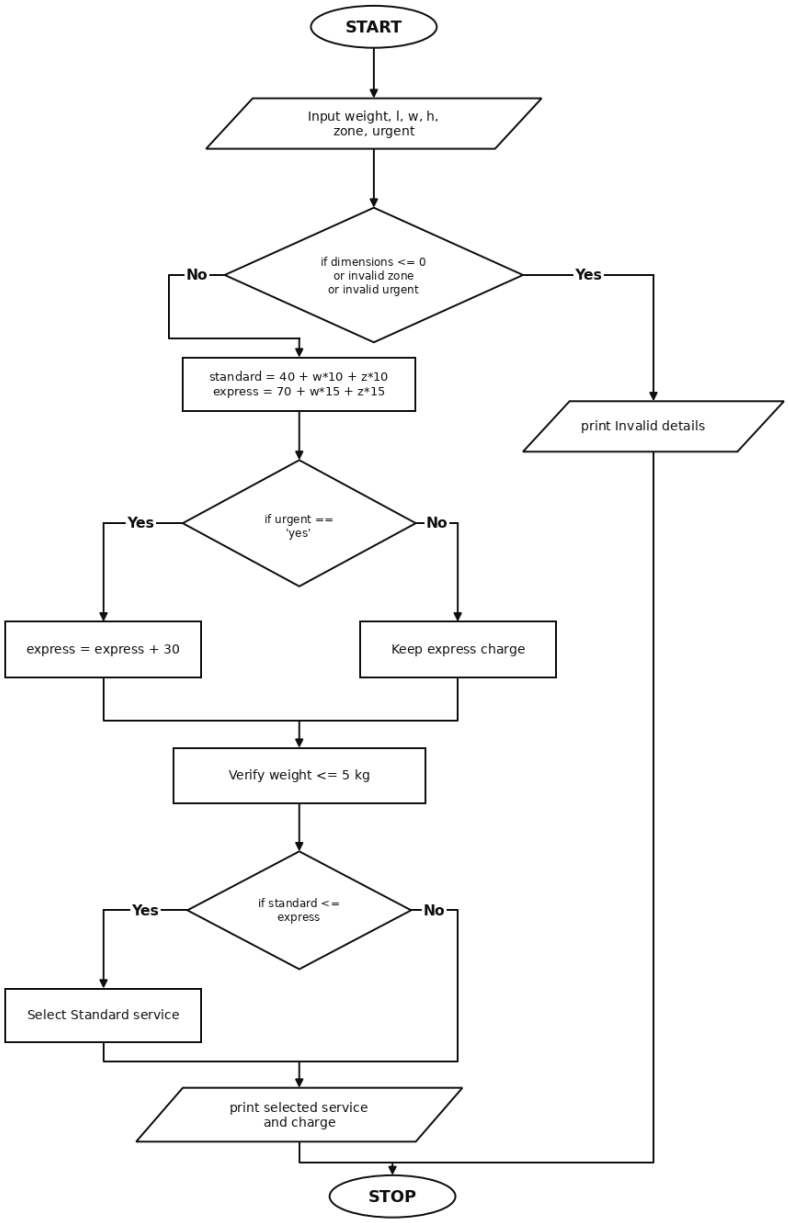
Assumptions

- Parcel weight, length, width, and height must all be positive values.
- Distance zones are restricted to 1, 2, or 3, and parcels above 5 kg are not serviceable.
- Urgent delivery adds a surcharge of Rs. 30 to the Express tariff.

Algorithm

1. Start.
2. Read parcel weight, length, width, height, distance zone, and urgency status.
3. Validate positive dimensions/weight, zone in [1, 2, 3], and urgent in ['yes', 'no'].
4. Calculate standard = 40 + weight * 10 + zone * 10.
5. Calculate express = 70 + weight * 15 + zone * 15 (plus 30 if urgent == 'yes').
6. If weight > 5, print 'No service available for this weight'.
7. Otherwise compare standard and express and print the lower-cost service and charge.
8. Stop.

Flowchart



Pseudocode

START

INPUT weight, length, width, height, zone, urgent

IF weight <= 0 OR length <= 0 OR width <= 0 OR height <= 0

    PRINT "Invalid parcel details"

ELSE IF zone NOT IN [1, 2, 3] OR urgent NOT IN ["yes", "no"]

    PRINT "Invalid zone or urgency option"

ELSE

```
    standard = 40 + weight * 10 + zone * 10
    express = 70 + weight * 15 + zone * 15
    IF urgent == "yes"
        express = express + 30
    IF weight > 5
        PRINT "No service available for this weight"
    ELSE IF standard <= express
        PRINT "Selected service: Standard", standard
    ELSE
        PRINT "Selected service: Express", express
STOP
```

Source Code

```
weight = float(input("Parcel weight in kg: "))
length = float(input("Length in cm: "))
width = float(input("Width in cm: "))
height = float(input("Height in cm: "))
zone = int(input("Distance zone (1-3): "))
urgent = input("Urgent delivery? (yes/no): ").strip().lower()

if weight <= 0 or length <= 0 or width <= 0 or height <= 0:
    print("Invalid parcel details")
elif zone not in [1, 2, 3]:
    print("Invalid distance zone")
elif urgent not in ["yes", "no"]:
    print("Invalid urgency option")
else:
    standard = 40 + weight * 10 + zone * 10
    express = 70 + weight * 15 + zone * 15

    if urgent == "yes":
        express = express + 30

    if weight > 5:
        print("No service available for this weight")
    elif standard <= express:
        print("Selected service: Standard")
        print("Charge: Rs.", round(standard, 2))
```

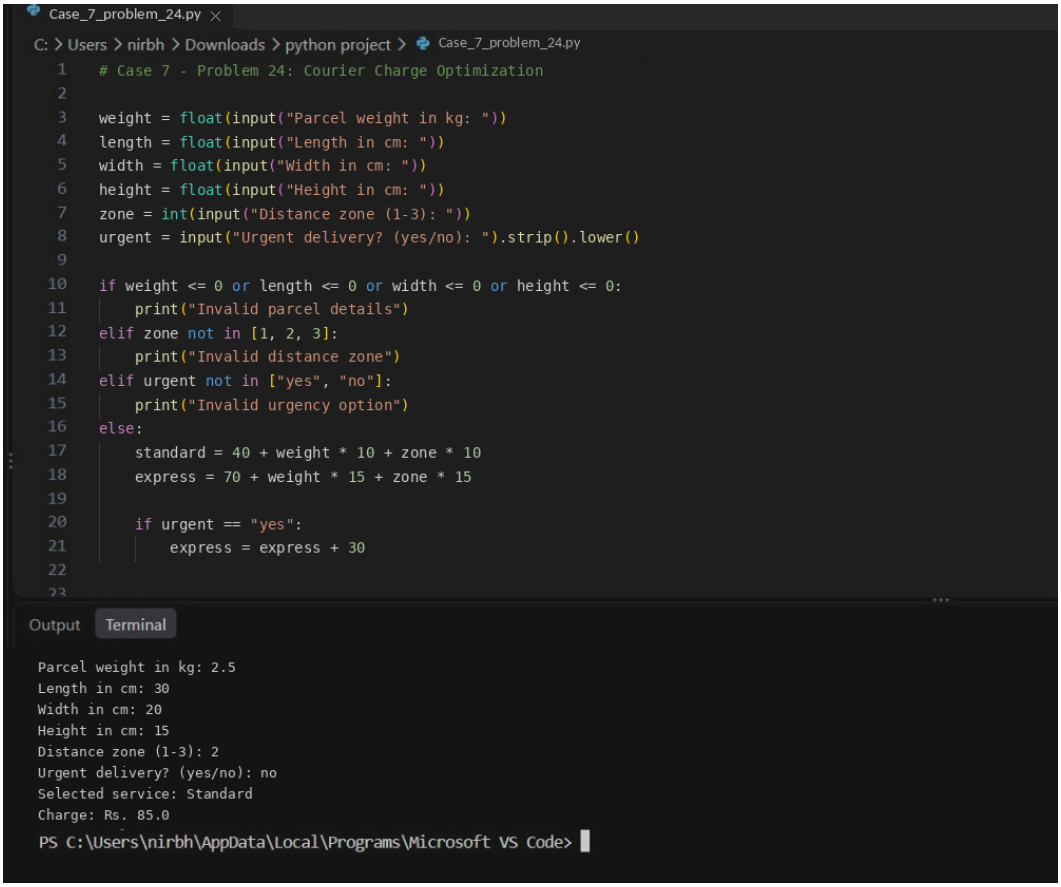
```
else:
    print("Selected service: Express")
    print("Charge: Rs.", round(express, 2))
```

Test Cases

1. Parcel weight in kg: 2.5
   Length in cm: 30
   Width in cm: 20
   Height in cm: 15
   Distance zone (1-3): 2
   Urgent delivery? (yes/no): no
   Output:
   Selected service: Standard
   Charge: Rs. 85.0

2. Parcel weight in kg: 6.0
   Length in cm: 40
   Width in cm: 30
   Height in cm: 25
   Distance zone (1-3): 1
   Urgent delivery? (yes/no): yes
   Output:
   No service available for this weight

Output



Conclusion

The program validates parcel parameters, computes zone-based courier charges, and selects the optimal delivery service within weight limits.

8.

Smart Canteen Order and Stock Management: Process multiple item orders against live stock levels, compute order bills, reject insufficient stock requests, and report daily sales and remaining inventory.

Sol:

Problem statement

Process sequential canteen orders using parallel item, price, and stock lists, deduct fulfilled quantities from stock, reject orders exceeding available inventory, and display total sales.

Objectives

- Maintain menu items, unit prices, and available stock using parallel lists.
- Validate item codes and requested quantities against live inventory for each order.
- Compute individual order bills, total daily sales, and closing stock balances.

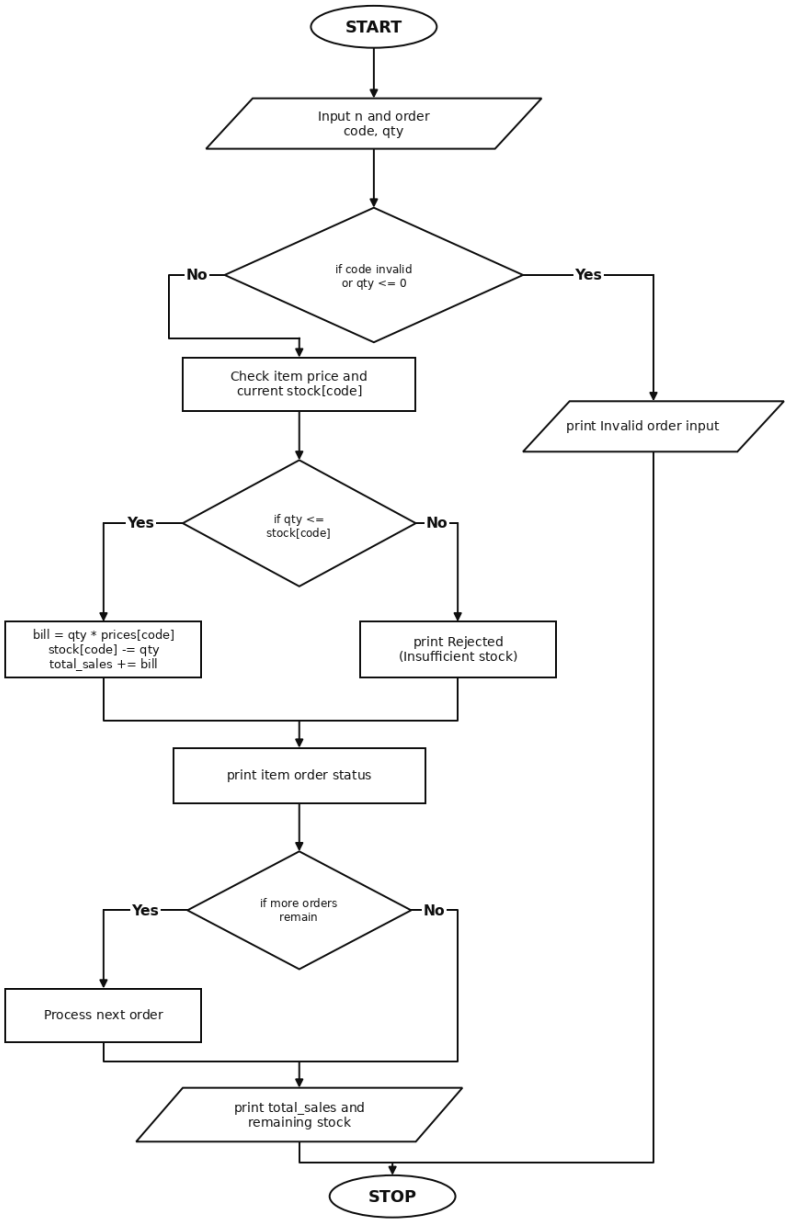
Assumptions

- Item codes are 0 for Samosa (Rs. 20), 1 for Sandwich (Rs. 45), and 2 for Coffee (Rs. 25).
- Initial stock levels are 30 Samosas, 15 Sandwiches, and 25 Coffees.
- Orders requesting more than available stock are rejected without partial fulfillment.

Algorithm

1. Start.
2. Initialize items, prices, and stock lists, and set total_sales = 0.
3. Read the total number of orders n.
4. For each order, read item code and quantity qty.
5. If code is out of range or qty <= 0, print 'Invalid order input'.
6. Else if qty <= stock[code], compute bill = qty * prices[code], deduct stock, and add bill to total_sales.
7. Else print that the order is rejected due to insufficient stock.
8. Display total daily sales and remaining stock for each item, then stop.

Flowchart



Pseudocode

START

SET items = ["Samosa", "Sandwich", "Coffee"]

SET prices = [20, 45, 25], stock = [30, 15, 25], total_sales = 0

INPUT n

FOR i = 1 TO n

  INPUT code, qty

  IF code < 0 OR code >= len(items) OR qty <= 0

```
        PRINT "Invalid order input"
    ELSE IF qty <= stock[code]
        bill = qty * prices[code]
        stock[code] = stock[code] - qty
        total_sales = total_sales + bill
        PRINT items[code], qty, bill
    ELSE
        PRINT items[code], "Rejected (Insufficient stock)"
PRINT total_sales and remaining stock
STOP
```

Source Code

```
items = ["Samosa", "Sandwich", "Coffee"]
prices = [20, 45, 25]
stock = [30, 15, 25]

total_sales = 0
n = int(input("Enter number of orders: "))

for i in range(n):
    print("Order", i + 1, "(0:Samosa, 1:Sandwich, 2:Coffee)")
    code = int(input("Enter item code: "))
    qty = int(input("Enter quantity: "))

    if code < 0 or code >= len(items) or qty <= 0:
        print("Invalid order input")
    elif qty <= stock[code]:
        bill = qty * prices[code]
        stock[code] = stock[code] - qty
        total_sales = total_sales + bill
        print(items[code], "x", qty, "-> Bill: Rs.", bill)
    else:
        print(items[code], "-> Rejected (Insufficient stock)")

print("Total Daily Sales: Rs.", total_sales)
print("Remaining Stock:")
for i in range(len(items)):
    print(items[i], ":", stock[i])
```

Test Cases

1. Enter number of orders: 2

   Order 1 -> Enter item code: 0, Enter quantity: 5

   Order 2 -> Enter item code: 1, Enter quantity: 2

   Output:

   Samosa x 5 -> Bill: Rs. 100

   Sandwich x 2 -> Bill: Rs. 90

   Total Daily Sales: Rs. 190

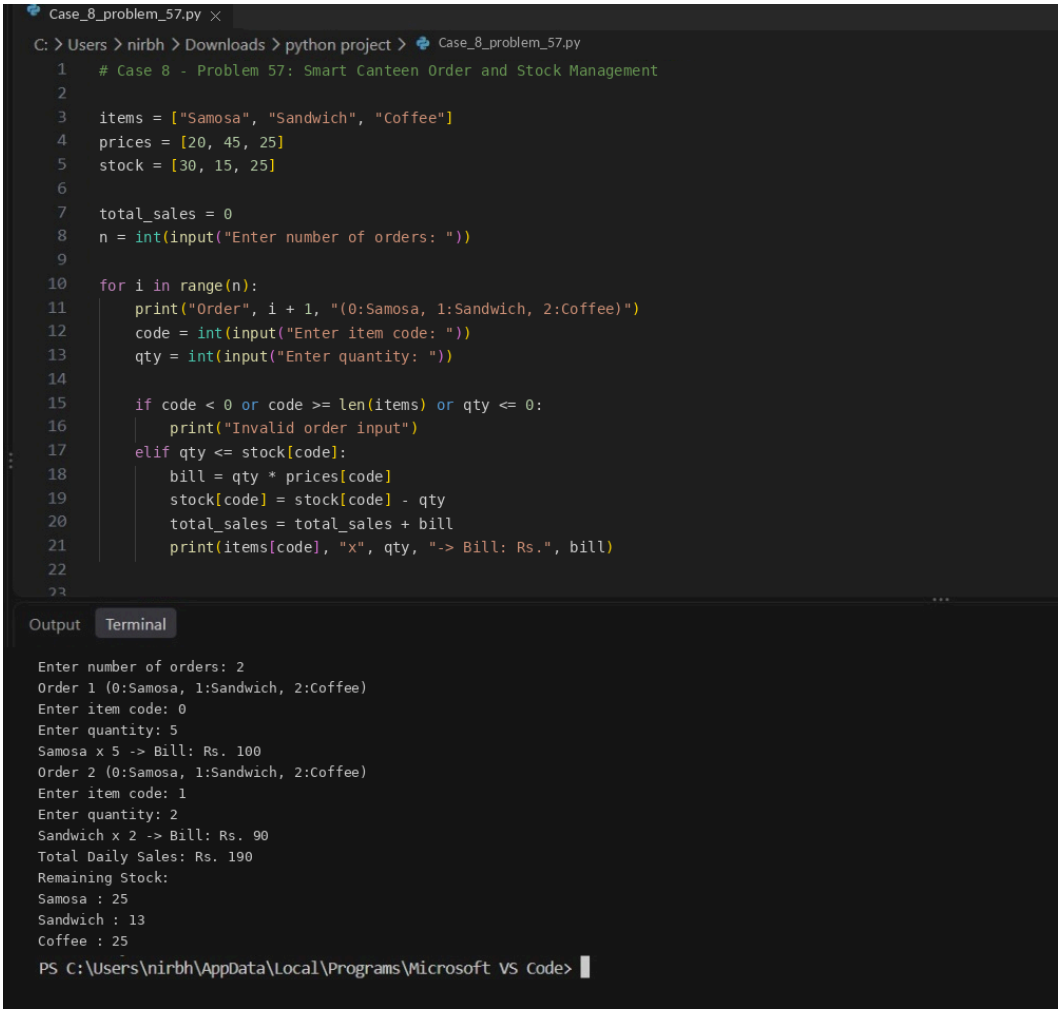
   Remaining Stock:

   Samosa : 25

   Sandwich : 13

   Coffee : 25

Output



Conclusion

The program processes canteen orders using parallel lists, updates live inventory, and prints daily sales and remaining stock.

9.

Campus Resource Allocation Under Constraints: Allocate limited projectors across sequential campus activities, track activity-wise allocations and equipment shortages, and display remaining inventory.

Sol:

Problem statement

Allocate available campus projectors across multiple student activities in order of request, calculate activity-wise shortages when demand exceeds supply, and report remaining projectors.

Objectives

- Read total available projectors and the number of scheduled campus activities.
- Allocate projectors up to the available inventory for each activity and compute any shortage.
- Display activity-wise allocation details, total equipment shortage, and remaining projectors.

Assumptions

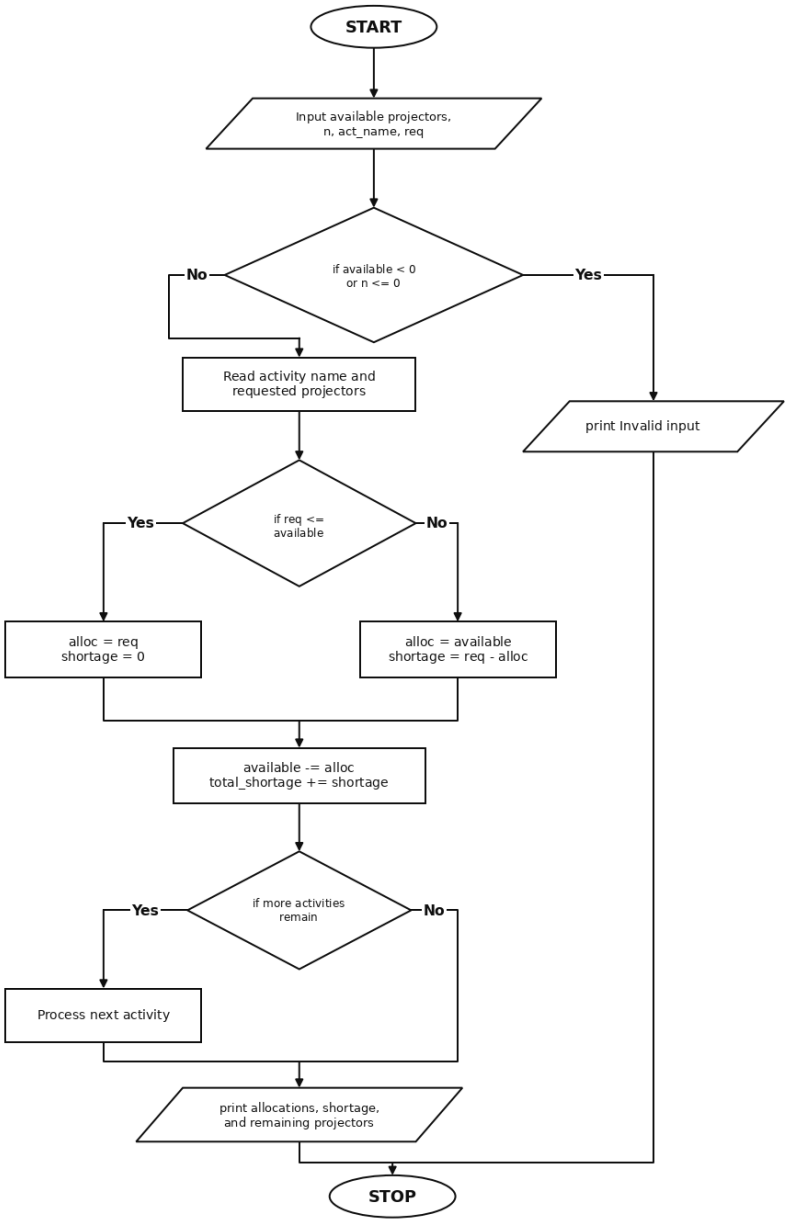
- Total available projectors must be non-negative and number of activities must be greater than zero.
- Projectors are allocated sequentially on a first-come, first-served basis.
- When requested projectors exceed available stock, all remaining projectors are allocated and the difference is recorded as shortage.

Algorithm

1. Start.
2. Read total available projectors and number of activities n.
3. Validate that available >= 0 and n > 0.
4. Initialize total_shortage = 0.
5. For each activity from 1 to n, read activity name and requested projectors req.

6. If req <= available, set alloc = req and shortage = 0; else set alloc = available and shortage = req - alloc.
7. Update available = available - alloc and total_shortage = total_shortage + shortage.
8. Display activity allocation summary, total shortage, and remaining projectors, then stop.

Flowchart



Pseudocode

START

INPUT available, n

```
IF available < 0 OR n <= 0
   PRINT "Invalid input"
ELSE
   SET total_shortage = 0
   FOR i = 1 TO n
      INPUT act_name, req
      IF req <= available
         alloc = req
         shortage = 0
      ELSE
         alloc = available
         shortage = req - alloc
      available = available - alloc
      total_shortage = total_shortage + shortage
      PRINT act_name, req, alloc, shortage
   PRINT total_shortage, available
STOP
```

Source Code

```
available = int(input("Enter Total Available Projectors: "))
n = int(input("Enter Number of Activities: "))

if available < 0 or n <= 0:
   print("Invalid input")
else:
   total_shortage = 0
   for i in range(n):
      act_name = input("Enter Activity " + str(i + 1) + " Name: ")
      req = int(input("Requested Projectors: "))

      if req <= available:
         alloc = req
         shortage = 0
      else:
         alloc = available
         shortage = req - alloc

      available = available - alloc
```

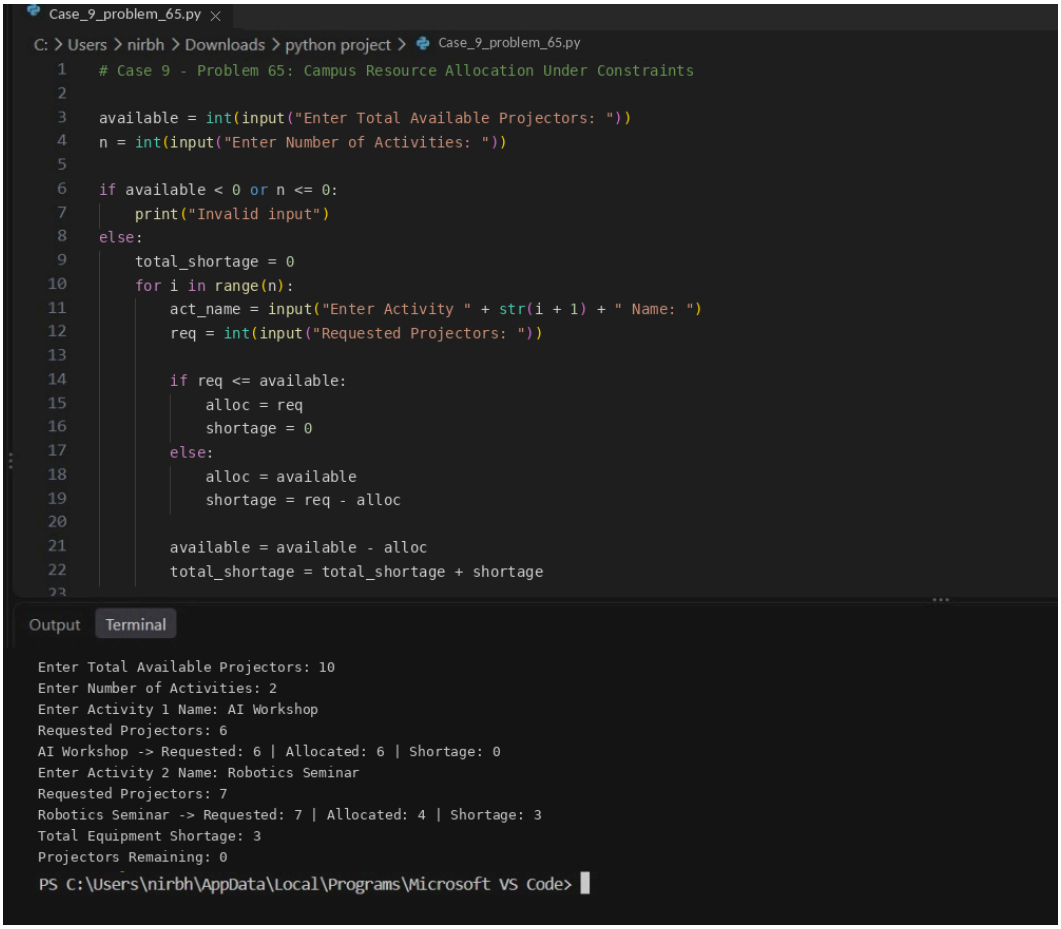
```
    total_shortage = total_shortage + shortage
    print(act_name, "-> Requested:", req, "| Allocated:", alloc, "| Shortage:", shortage)

print("Total Equipment Shortage:", total_shortage)
print("Projectors Remaining:", available)
```

Test Cases

1. Enter Total Available Projectors: 10
   Enter Number of Activities: 2
   Enter Activity 1 Name: AI Workshop
   Requested Projectors: 6
   Enter Activity 2 Name: Robotics Seminar
   Requested Projectors: 7
   Output:
   AI Workshop -> Requested: 6 | Allocated: 6 | Shortage: 0
   Robotics Seminar -> Requested: 7 | Allocated: 4 | Shortage: 3
   Total Equipment Shortage: 3
   Projectors Remaining: 0

Output



Conclusion

The program allocates constrained campus projector inventory across activities, tracks equipment shortages, and displays remaining stock.

10.

Campus Festival Registration and Financial Audit: Filter duplicate student registrations, compute unique registration revenue and total festival income with sponsorship, and audit the final financial balance.

Sol:

Problem statement

Identify unique and duplicate student registration IDs from a participant list, compute registration revenue and total income including sponsorship, and calculate the net financial balance after expenses.

Objectives

- Filter participant IDs into unique registrations and duplicate entries using list membership checks.
- Calculate registration revenue from unique participants and add sponsorship funds.
- Compute and display the final financial balance after deducting total festival expenses.

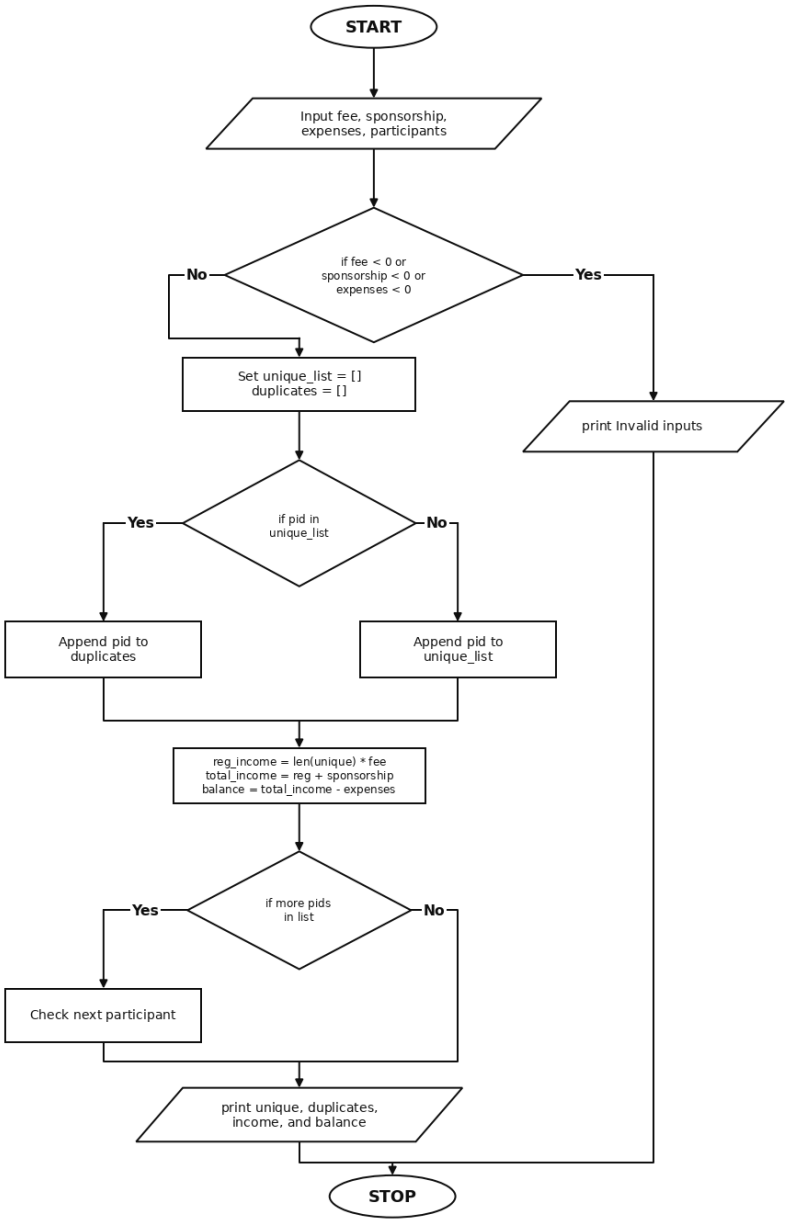
Assumptions

- Registration fee, sponsorship amount, and total expenses must be non-negative.
- Duplicate student IDs are excluded from registration fee calculations.
- Final financial balance equals total festival income minus total festival expenses.

Algorithm

1. Start.
2. Read registration fee per student, sponsorship amount, and total festival expenses.
3. Load the list of participant student IDs.
4. Validate that fee >= 0, sponsorship >= 0, and expenses >= 0.
5. Iterate through participants: if ID is in unique_list, append to duplicates; else append to unique_list.
6. Compute reg_income = len(unique_list) * fee, total_income = reg_income + sponsorship, and balance = total_income - expenses.
7. Display unique participant count, duplicate entries, revenues, and final balance, then stop.

Flowchart



Pseudocode

START

INPUT fee, sponsorship, expenses

IF fee < 0 OR sponsorship < 0 OR expenses < 0

  PRINT "Invalid financial inputs"

ELSE

  SET unique_list = [], duplicates = []

  FOR each pid IN participants

```
        IF pid IN unique_list
            Append pid to duplicates
        ELSE
            Append pid to unique_list
    reg_income = len(unique_list) * fee
    total_income = reg_income + sponsorship
    balance = total_income - expenses
    PRINT len(unique_list), duplicates, reg_income, total_income, balance
STOP
```

Source Code

```
fee = float(input("Enter Registration Fee per Student: "))
sponsorship = float(input("Enter Sponsorship Amount: "))
expenses = float(input("Enter Total Festival Expenses: "))

participants = ["STU101", "STU102", "STU103", "STU101", "STU104", "STU102"]

if fee < 0 or sponsorship < 0 or expenses < 0:
    print("Invalid financial inputs")
else:
    unique_list = []
    duplicates = []

    for pid in participants:
        if pid in unique_list:
            duplicates.append(pid)
        else:
            unique_list.append(pid)

    reg_income = len(unique_list) * fee
    total_income = reg_income + sponsorship
    balance = total_income - expenses

    print("Unique Participants:", len(unique_list))
    print("Duplicate Entries Detected:", duplicates)
    print("Registration Revenue: Rs.", round(reg_income, 2))
    print("Total Festival Income: Rs.", round(total_income, 2))
    print("Final Financial Balance: Rs.", round(balance, 2))
```

Test Cases

1. Enter Registration Fee per Student: 500

   Enter Sponsorship Amount: 15000

   Enter Total Festival Expenses: 12000

   Output:

   Unique Participants: 4

   Duplicate Entries Detected: ['STU101', 'STU102']

   Registration Revenue: Rs. 2000.0

   Total Festival Income: Rs. 17000.0

   Final Financial Balance: Rs. 5000.0

2. Enter Registration Fee per Student: 250

   Enter Sponsorship Amount: 5000

   Enter Total Festival Expenses: 6500

   Output:

   Unique Participants: 4

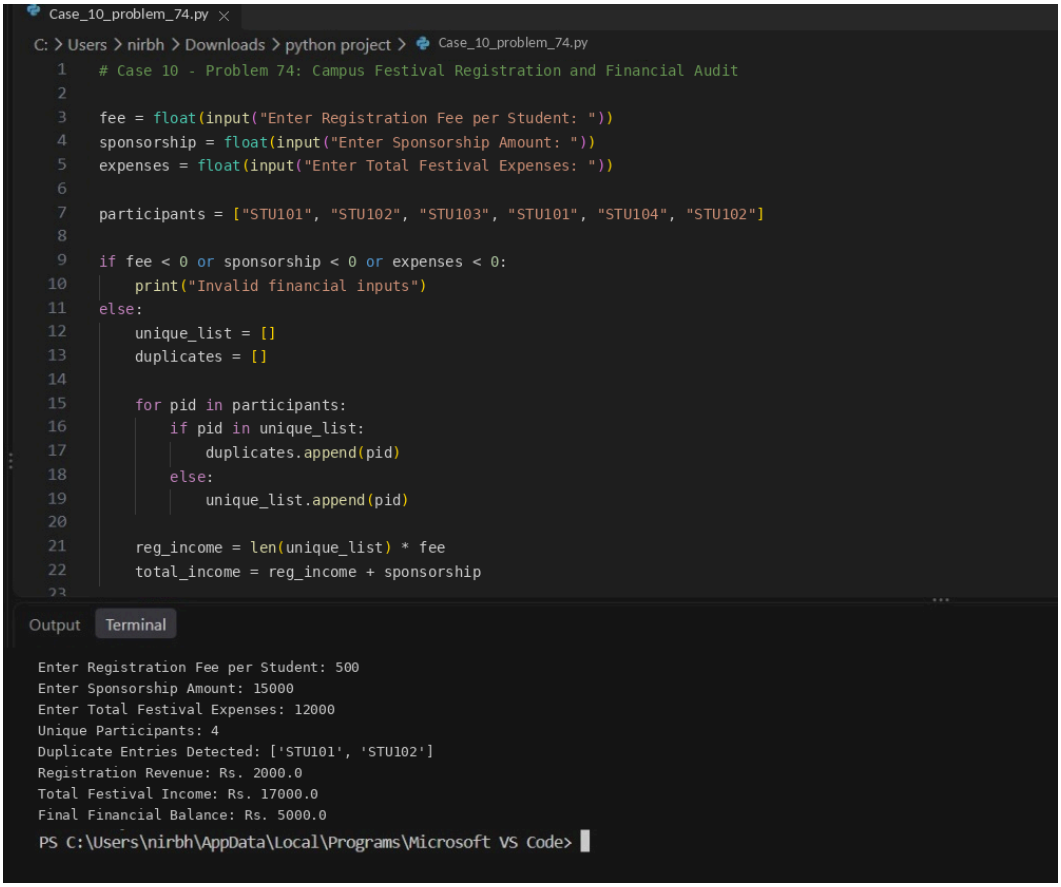
   Duplicate Entries Detected: ['STU101', 'STU102']

   Registration Revenue: Rs. 1000.0

   Total Festival Income: Rs. 6000.0

   Final Financial Balance: Rs. -500.0

Output



Conclusion

The program removes duplicate student registrations, computes total festival revenue with sponsorship, and audits the final balance.